V2 Launch Checklist

> **The single list to read before shipping.** Open work was scattered across `DEBT_TRACKER.md`, four other planning docs and ~20 memory files, so "are we ready?" could not be answered. This file answers it.
>
> **Rules.** Every claim cites `file:line` — a claim without evidence gets deleted, not debated. This file **links to** `DEBT_TRACKER.md` rather than restating it; the tracker stays the record of *why* something is owed, this is the record of *what blocks launch*. Re-verify before believing any row: rows rot faster than code.

Target: limited pilot (TestFlight / friends). Decided 2026-08-11.

Gate 0 — the paid Apple Developer Program ($99/yr)

Not one item among many. **Three of the four newly-scoped features sit behind it**, and so does the distribution channel itself.

Blocked thing	Why
Push notifications	`plugins/withoutPushEntitlement.js:19` exists *only* to strip `aps-environment`, which a personal team cannot sign. Deleting the plugin is the small part.
App Intents	Native Swift target + entitlements.
Widget	A widget extension is a second signed target.
TestFlight external testing	Needs App Store Connect.

- [ ] **Buy it.** Everything below assumes it. Without it, the voice rework degrades to in-app mic only — which still works, and still removes the Shortcuts round trip.

1. Ship blockers — fix before anyone else installs

- [ ] **Receipt-OCR proxy is not in this repo.** `src/lib/ocrProviders/gemini.ts` calls `EXPO_PUBLIC_RECEIPT_OCR_PROXY_URL`; `.env` is gitignored and `server/receipt-ocr-proxy/` lives elsewhere. If it is unset in the release build, **Scan degrades silently** for every pilot user. Deploy it, set the var, verify.
- [ ] **`expo-file-system/legacy` may already throw at runtime in SDK 56.** `src/lib/avatar.ts` and `ocrProviders/gemini.ts` still call it, while `src/lib/pdfjsCache.ts` states the legacy API throws. Jest stubs the module, so the green suite proves nothing. **Device check.**
- [ ] **`category_global_v1` has never run against a real existing DB** (`src/db/schema.ts:523`). It drops and rebuilds the `category` table. Test against a copy of a populated database before it runs on anyone's phone.
- [ ] **Demo/seed data must not ship enabled.** `seedDemo.ts` plus the CSV export's hardcoded demo-row signatures drift apart by design.
- [ ] **Privacy policy + App Store listing** — required even for external TestFlight.
- [ ] **App icon, splash, screenshots** — never audited.
- [ ] **Pass 4 was never device-tested**, and the `splitting` flag (`src/lib/featureFlags.ts:47`) changes the tab bar.

2. Open decisions — these need your call, not more analysis

D1. What does a group budget mean? *(deferred here deliberately)*

The evidence. `category_budget` is stored **group-wide** — no `person_id` column (`src/db/schema.ts:128–135`). But it is measured **against your share only**: `getCategorySpending` picks `t.shares.find(sh => sh.personId === meId)?.amount` (`src/lib/budget.ts:107–109`), and the group screen always passes you (`app/group/[id].tsx:79`).

So a **₹10,000 Groceries limit on a 4-person flat is silently compared against your quarter.** Whatever you intended, that is what ships today.

Option	Cost	Consequence
Relabel as "my share"	Label only — the engine already does this	Consistent with Home + Insights, which moved to my-share. Cannot express "₹10k for the flat".
Make it group-whole	Stop filtering to `meId`	Matches how most people read the number. Changes every group budget figure now on screen.
Store both	New per-person dimension on `category_budget`	Most informative, most work, most schema.

Recommendation: relabel now, revisit group-whole when multi-user is real.

D2. Do investments/crypto get a pay method?

There is no crypto concept anywhere — grep finds nothing. `investments` is a single manually-entered figure, read once and clamped (`src/lib/cash.ts:127`); no transaction ever touches it. Either it stays manual, or "move money to investments" becomes a Transfer kind. Crypto would be **new**, not an extension.

D3. Health engine still uses group-total on both sides

It pairs group-total budget with group-total spend while everything else moved to my-share (`docs/V2_FIX_PLAN.md:374`). Deliberately not rebased. Resolve alongside **D1** — they are the same question.

D4. Monetisation shape

No paywall, entitlement or purchase SDK exists, and none of it is reusable from what does. Out of scope for the pilot; the decision is *when*, not *whether*.

3. Newly scoped for V2

3.1 Voice — mic, not a text field filled by speech

The current design is the problem, not a bug in it. Siri's `Ask for Input` *is* a text field; that is exactly why it feels like a second input.

Only the capture layer changes. `parseVoice` (`src/lib/voiceParse.ts`), `detectVoiceKind` , `routeVoiceDraft` (`src/lib/voiceInbox.ts`) and the auto-save path (`app/add/quick.tsx`) all take a plain string. Every bit of it is reused.

- [ ] **In-app mic button** — on-device recognition, live partial transcript, insert on silence. Needs a native module: **there is no first-party Expo speech-to-text** (`expo-speech` is text-to-*speech*, and is not even installed). Precedent for a local module exists — `modules/expo-ocr` .
- [ ] **App Intents** — true hands-free, no app launch, Siri reads the result back. This is what `DEBT_TRACKER.md:307` has been deferring all along.
- [ ] **Retire on landing:** `voiceDrain.ts` (already marked legacy), `voiceShortcut.ts` , `voiceShortcutFile.ts` , `scripts/build-shortcuts.ts` , and the import→share→paste round trip that has now cost **six** passes.

3.2 Money — pay method moves the right balance *(decided: per-method baselines)*

The enum mixes rails with accounts. `PayMethod` (`src/constants/enums.ts:59`) lists `upi` beside `cash` and `bank` — but UPI is a *way to move money out of a bank account*, not a place money sits. Giving it its own pot would double-count.

Pot	Fed by	Baseline
Cash	`cash`	`money.opening_cash` *(exists)*
Bank	`bank` , `upi` , `autopay` , `other`	`money.opening_bank` *(new)*
Wallet	`wallet`	`money.opening_wallet` *(new)*
Credit	`card`	`money.credit_used` *(exists — already works exactly this way)*

The pattern is already proven in production. `card` stores a baseline and derives the delta since `money.updated_at` (`src/lib/cash.ts:129–139`, `src/db/queries/cashQuery.ts:32–48`), so edits and deletes self-correct and there is no second ledger. This is that same shape, three more times.

- [ ] **It also makes `INCOME_LANDING` real.** Income already asks "Landed in ___" (`src/constants/enums.ts:85`) and **nothing reads the answer** — `src/lib/cash.ts:81` treats every income identically.
- [ ] Touches `moneyProfile.ts`, `cashQuery.ts`, `cash.ts`, `MoneyEditorSheet.tsx`, `TotalMoneyCard.tsx`, and the Onboarding money step.
- [ ] **Card repayment is still unmodelled** (`DEBT_TRACKER.md:71`) — `creditUsed` only grows between Plan edits. Decide whether the pilot ships with that.

3.3 Push, widget, proper import structure

- [ ] **Push** — delete `plugins/withoutPushEntitlement.js` once Gate 0 clears. Only local notifications exist today.
- [ ] **Widget** — new signed target. Scope it: balance? today's spend? quick-add?
- [ ] **Import structure** — `app/import.tsx` + `paytmParse.ts` + `pdfjsCache.ts`. pdf.js is pinned to a **CDN URL fetched on first use**; bundle it locally so import works offline and does not depend on someone else's uptime.

3.4 WhatsApp payment reminders — feasible, but not the automatic version

Wanted: keep people's numbers and nudge everyone who owes you, on WhatsApp, free.

Half of it already exists. `person.mobile` is a real column (`src/db/schema.ts:21`, migration `:224`) that **nothing reads or writes** — `insertPerson` hard-codes `null` (`src/db/queries/persons.ts:52`). It is dead schema today, like `remote_uid`. And `getMyExposure` (`src/db/queries/balances.ts:136`) already returns the per-person netted balance, so "who owes what" needs no new maths.

The free path — `wa.me`, and it is genuinely free. `https://wa.me/<number>?text=<encoded>` opens WhatsApp on a chat with the message pre-filled. It is *your* WhatsApp account sending your own message; there is no API, no account, no cost, and no backend. Needs no new dependency — `expo-linking` is installed.

⚠️ **What is not possible: silent send-to-all.** Two independent walls, and the second is the one that matters:

1. A deep link opens **one chat** and always needs a manual tap to send. WhatsApp prevents third-party apps from sending on your behalf, deliberately.
2. The programmatic route is the **WhatsApp Business Cloud API**, which needs a Meta Business account, a *separate business phone number* (not your personal one), pre-approved message templates, and **a backend server** — this app is local-first and has none. It is also **not free**: since 1 Jul 2025 Meta charges per *delivered template message*, ~₹0.115–0.145 for utility messages in India. Free only inside an already-open customer-service window, which a debt reminder is not. Meta's policy also prohibits unsolicited bulk messaging.

Bulk — one send, private 1:1 delivery, no group — is a WhatsApp Broadcast List. One message goes out as **separate one-to-one chats**; each person sees a private message and replies only to you. Free, up to **256 per list**, unlimited lists.

Two constraints, and the first is a silent failure:

1. **Recipients must have your number saved in their contacts.** If they haven't, they simply never receive it, with no error on either side. Usually true of flatmates; not guaranteed of a one-off trip group.
2. **Every recipient gets identical text.** A broadcast cannot say "you owe ₹450" to one person and "₹1,200" to another.

That second point is the crux: personalisation is the thing that costs money. Free bulk means one message for everyone; per-person amounts mean either N sends or the paid Cloud API. There is no free, one-tap, personalised route — that combination does not exist.

Route	One send?	Private 1:1?	Personalised?	Cost
WhatsApp Broadcast List	✅	✅	❌ same text	**Free**
`mailto:?bcc=`	✅	✅	❌ same text	Free — but nobody reads email for ₹450
Per-person `wa.me`	❌ N sends	✅	✅	Free

Route	One send?	Private 1:1?	Personalised?	Cost
Multi-recipient SMS	✅	⚠️ usually becomes a **group MMS** thread	❌	Carrier rates
WhatsApp Cloud API	✅	✅	✅	~₹0.115–0.145/msg + backend — rejected

Recommendation: a generic broadcast nudge carrying your VPA — "Settle up on BudgetSplit, pay me at prem@okhdfc" — with the amounts staying in the app. It is free, private, and one action; the amounts were never the part that needed to travel.

- [ ] ⚠️ **Device-check first: can the system share sheet target a Broadcast List?** WhatsApp's picker shows chats and groups; whether broadcast lists appear there is **unverified**, and the whole recommendation rests on it. If not, the user composes in WhatsApp and we only supply the text via copy-to-clipboard.
- [ ] **Compose the nudge** from `getMyExposure` (`src/db/queries/balances.ts:136`) and hand it to `Share.share` . Needs no stored numbers and no permissions — a reason to build this before the phone field.
- [ ] Add a phone field beside the UPI ID in the Friends rename sheet (`app/friends.tsx:81`) — makes the dead `mobile` column live. Only needed for the **per-person** path.
- [ ] Per-person **Remind on WhatsApp** as the secondary path, pre-filled with that person's amount and your VPA as payable text. (A `upi://` link is **not** tappable inside WhatsApp — send the handle as text.)
- [ ] Multi-recipient SMS needs **platform-specific syntax**: iOS `sms://open?addresses=a,b&body=…` , Android `sms:a,b?body=…` . Apple does not implement RFC 5724's comma list, so the obvious form silently fails on iPhone. Note it usually lands as a **group** thread, so it does not satisfy "bulk, not group".
- [ ] **A cooldown, and never auto-send.** A reminder feature with no floor becomes a way to annoy your friends daily. Record `last_nudged_at` ; grey the button inside it.

The framing that makes this work — decided 2026-08-11

The stated problem is not typing, it is **that some people are bad at asking for money**. Automation was never the fix, and true automation is not available for free anyway (the

manual tap *is* WhatsApp's anti-spam mechanism). Two cheap moves address it directly:

1. **The app initiates, the user confirms.** "*Rohan has owed ₹450 for 12 days. Remind?*" → one tap → pre-filled. You never decide to chase anyone; you agree with the app. That moves the social burden off the user, which is the actual ask.
2. **The app is the author, not the user.** "*Sent from BudgetSplit — ₹450 from the Goa trip*" reads as a receipt; "*hey can you send me 450*" reads as a demand. Identical information, completely different social act. Costs nothing.

Build order — the two halves are not the same size:

- [ ] **In V2 · the composer + manual tap.** Depersonalised line from `getMyExposure` → `Share.share`, from any balance row. No schema, no numbers, no permission, no new dependency. Small.
- [ ] **After the pilot · the scheduled nudge.** Needs an overdue scan, a per-person cooldown store, notification routing, and a cadence that nudges without nagging — **and that cadence cannot be guessed from an empty pilot.** Get it wrong and users disable notifications, which loses the channel permanently. The manual composer is a strict prerequisite, so nothing is wasted by waiting.

⚠️ **In V2, but not ahead of §1.** This blocks nobody; the three §1 items break things for real pilot users.

- [ ] `sms:` fallback for people without WhatsApp, and the system share sheet for everyone else — same pre-filled string, three transports.

Who pays what — the whole point is that we never do. Every transport below is a *deep link*: we hand a pre-filled message to an app the user already has, and they send it as themselves. There is no server in the path, so there is no per-message bill to us, ever.

WhatsApp is free in every shape — broadcast or per-person — because it is their own account sending. `sms:` is the only transport that costs the *user* anything (their carrier's rate, usually inside a bundled Indian plan), so it should be labelled "standard SMS rates" rather than implied free. The comparison of routes is in the table above.

- [ ] **Say the privacy line out loud.** Nothing leaves the device: the text is handed to WhatsApp, and there is no server in the path. That is worth stating given the app's standing promise.
- [ ] **Decide: import from Contacts, or type numbers by hand?** `expo-contacts` is not installed, and asking for the whole address book is a heavy permission for a pilot.

Recommendation: manual entry first.

4. UI/UX rework — with the cause, not just the symptom

4.1 Review header truncation *(measured)*

`ReviewSourceTabs.tsx:40` builds `` `${TXN_SOURCE_LABEL[src]} ${countOf(src)}` `` — e.g. `"Said out loud 12"` — and hands it to `TabPills`, a **fixed equal-flex, non-scrolling** segmented control (`TabPills.tsx:83`: `pillW = track / tabs.length`).

On a 393pt screen with 4 pills that is **~89pt per pill**; the label needs **~108pt** at 13px SemiBold. And because the count is **last in the string, the count is what gets cut** — the one thing being scanned.

- [ ] Short-form labels + count as a separate badge, or a scrollable `TabPills` variant.
- [ ] Saved-view banner one-line clips the count *and* payer (`review.tsx:513`).
- [ ] Footer CTA wraps and breaks the 52pt button height (`review.tsx:629`).
- [ ] Filter chips hard-capped at `maxWidth: 160` (`FChip.tsx:18`) — the "Househol…" pattern.

4.2 Insights x-axis truncation *(cause found)*

`app/insights.tsx:148` sets `spacing={Math.max(8, 300 / len)}`. One point per day means a 31-day month gives **9.68px per label container**, and the chart library renders each label in a `View` exactly `spacing` wide at `numberOfLines={1}`. One digit fits; two do not — hence `"1…"`, `"2…"`, `"3…"`.

- [ ] Measure the container via `onLayout` — `300` is a hardcoded magic width and the chart never measures anything.
- [ ] Or bucket weekly: 4–5 fat points instead of 31 hairlines.

4.3 Insights — ten equal cards, and two real bugs inside them

Everything is a card in one `ScrollView` with an 8px gap (`insights.tsx:327`), so nothing is more important than anything else.

- [ ] 🐞 **Two different forecast models print different numbers ~100px apart.** The hero uses a naive run-rate — `Math.round((monthSpend / dayOfMonth) * daysInMonth)` (`insightsData.ts:69`) — while the chart badge uses the credibility-weighted model (`src/lib/forecast.ts:43`).
- [ ] 🐞 **The savings nudges are random** — `generateInsights(ctx, maxN = 3, rng = Math.random)` (`savingsInsights.ts:28`) reshuffles on every pull-to-refresh. An insight you cannot return to is not an insight.
- [ ] **The month pill is a fake control** — `insights.tsx:79` renders a pill-shaped muted `View` with **no `onPress`** . Wire it or make it a plain label.

Proposed structure — three tiers instead of ten peers:

1. **One headline, always present.** Today it renders *only* when overspending (`insights.tsx:99`), so a good month opens on a chart with no verdict. One sentence, one number, one action.
2. **Fact** — what happened: drivers, shifts vs last month.
3. **Forecast** — visually distinct (reuse the dashed treatment), **one model only**. Fold "What if…" in here; it is the same kind of claim.

Rows 6/7/8/10 are four hand-written variants of one row — collapse them.

4.4 Goals — cluttered list, and a sweep with real defects

Ten pieces of information per card (`GoalCard.tsx:42-96`), two of which are **the same number twice** (`p.pct` in the bar and again in the meta).

The engine is worse. In `src/lib/savingsEngine.ts` :

- [ ] 🐞 **`priority` is dead code.** `rankKey = g.sort_order ?? PRIORITY_RANK[g.priority]` (`:45-46`), but `sort_order` is `INTEGER NOT NULL DEFAULT 0` (`src/db/schema.ts:167`) — the fallback can never run.
- [ ] 🐞 **Before anyone drags, every goal has `sort_order = 0`** , so ties break on list order and **the newest goal is both funded first and raided first**. Funding and raiding are meant to be mirror images; they only become so after a manual drag.
- [ ] 🐞 **Reorder writes an incomplete permutation.** `savings.tsx:181` passes only *active* goals to the drag list, and `reorderGoals` writes `sort_order = i` for just those ids (`savings.ts:106-110`) — completed goals keep stale values and interleave.

- [ ] 🐞 **A completed goal can be raided.** The filter checks `locked` and `saved > 0`, never `saved >= target` (`savingsEngine.ts:104`).
- [ ] 🐞 **The raid prompt hides the amounts** — `savings.tsx:127` lists goal names only, though `withdrawals` carries `amount`. You approve a raid without seeing ₹.
- [ ] **There is no surplus sweep at all.** Nothing pushes an underspent month into goals; the only automatic inflow is the fixed per-goal allocation.

Proposal: one number per row (progress), detail on tap; **separate "fund first" from "protect from raid"** instead of overloading one drag axis; show ₹ per goal in the raid prompt; add the surplus sweep as an explicit opt-in.

4.5 Scan & Pay

- [ ] **QR from a photo needs no new dependency.** `expo-camera` ships `scanFromURLAsync` (**called nowhere** — grep returns 0) and `expo-image-picker` is already installed for receipts. Both scanners are live-camera only (`ScanPaySheet.tsx:210`, `UpiQrScanner.tsx:56`). Cheap win.
- [ ] **Why the UPI ID is kept:** it is a *settlement identity* — theirs to pay, yours to be paid (`person.upi_vpa`, read by `TransferBody.tsx:97` and the request-QR flow). Scan & Pay does **not** use it; that payee VPA is never persisted.
- [ ] **Say the quiet part on screen.** PhonePe, Paytm, Amazon Pay and WhatsApp reject externally-built intents (⛔ `F8`), so for those the "payment option" is cosmetic — the app opens bare and you re-enter the amount.

5. Device verification — nothing here is provable from a green test suite

- [ ] **Android has never run the UPI path at all** (`F9`). `useUpiApps` returns `null`, so no per-app finding on record applies to it.
- [ ] **Google Pay — the largest UPI app — has never been tested** on iOS (`gpay://` vs `tez://`).
- [ ] 5 iOS schemes are `provenance: 'unverified'` (`upiIntent.ts:235-282`) — a wrong path drops the payee.
- [ ] `emvQr.ts` was written from the EMVCo spec, **never validated against a real QR**.
- [ ] `detectVoiceKind` has never seen real `en-IN` dictation.
- [ ] Review's saved views / filters / bulk actions — built, never device-tested.

- [ ] The three release-gated items in `V2_FIX_PLAN.md:387` (UPI button, household persona card, backup row).

6. Known-and-accepted for the pilot

Not blockers. Listed so nobody re-discovers them as bugs.

- Attachment and avatar files **orphan forever** on delete (`DEBT_TRACKER.md:291`); only a manual "Delete all receipt photos" exists.
- `openDB()` re-runs ~40 `ALTER` s every launch (`DEBT-01`).
- `PRAGMA foreign_keys` is **OFF**; cascades are hand-rolled.
- Voice auto-save has no off switch — the guard is Undo plus the duplicate prompt.
- Files over the ~300-line rule: `review.tsx` 749, `Onboarding.tsx` 793, `itemized.tsx` 614.
- 45 files still import the `src/constants/*` shims instead of `src/theme` .

7. Explicitly out of scope — dropped, not deferred

Thing	Why
Gmail live ingestion / CASA	Restricted scope, Tier-3 assessment, ~$thousands/yr. The paste + file import path stays.
Account Aggregator	Needs a partner. Not started, not scoped.
GPay auto-import	Blocked on an unknown export format (`F4`).
Android SMS reading	Play-policy dead.
Multi-currency	Needs historical rates; get it right on day one or not at all.
Real multi-user sync	`person.remote_uid` is dead schema; there is no cloud.